

Web Application Development

Lab 7: SPRING BOOT & JPA CRUD

Part B: Homework

Task 5.1:

Implementation Details:

Repository Layer (ProductRepository)

- Added a custom query method searchProducts using the @Query annotation.
- JPQL query uses named parameters with conditional checks (:param IS NULL OR ...) to allow optional filtering.
- Query structure:

```
@Query("SELECT p FROM Product p WHERE " +  
        "(:name IS NULL OR p.name LIKE %:name%) AND " +  
        "(:category IS NULL OR p.category = :category) AND " +  
        "(:minPrice IS NULL OR p.price >= :minPrice) AND " +  
        "(:maxPrice IS NULL OR p.price <= :maxPrice)")  
List<Product> searchProducts(@Param("name") String name,  
                             @Param("category") String category,  
                             @Param("minPrice") BigDecimal minPrice,  
                             @Param("maxPrice") BigDecimal maxPrice);
```

Service Layer (ProductService & ProductServiceImpl)

- ProductService interface defines the method searchProducts(String name, String category, BigDecimal minPrice, BigDecimal maxPrice).
- Implementation in ProductServiceImpl delegates the search to the repository:

```
@Override  
public List<Product> searchProducts(String name, String category, BigDecimal minPrice, BigDecimal maxPrice) {  
    return productRepository.searchProducts(name, category, minPrice, maxPrice);  
}
```

Controller Layer (ProductController)

- Added an endpoint for advanced search:

```
// Advanced search
@GetMapping("/advanced-search")
public String advancedSearch(
    @RequestParam(required = false) String name,
    @RequestParam(required = false) String category,
    @RequestParam(required = false) BigDecimal minPrice,
    @RequestParam(required = false) BigDecimal maxPrice,
    Model model) {

    List<Product> results = productService.searchProducts(name, category, minPrice, maxPrice);
    model.addAttribute("products", results);
    model.addAttribute("name", name);
    model.addAttribute("category", category);
    model.addAttribute("minPrice", minPrice);
    model.addAttribute("maxPrice", maxPrice);

    return "product-list";
}
```

View Layer (product-list.html)

- Added a search form with fields for Name, Category, Min Price, and Max Price.
- Uses Thymeleaf th:action and th:value to bind form submission to the controller and pre-fill values.
- Results are displayed in the existing products table, and empty states are handled.

Task 5.2:

Implemented a category filter dropdown to allow users to view products by specific categories.

- **Repository:** Added a query to fetch all unique categories:

```
@Query("SELECT DISTINCT p.category FROM Product p ORDER BY p.category")
```

```
List<String> findAllCategories();
```

- **Service:** Added getAllCategories() to expose the list of categories to the controller.
- **Controller:** Populated the dropdown with categories and filtered products when a category is selected.

- **View:** Added a <select> dropdown in product-list.html that triggers filtering on change.

Task 5.3:

Pagination only works for search function, I created a separate search bar that takes keyword as input (name of product)

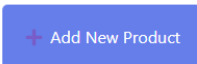
```

130
131 • @GetMapping("/search")
132 public String searchProducts(
133     @RequestParam("keyword") String keyword,
134     @RequestParam(defaultValue = "0") int page,
135     @RequestParam(defaultValue = "10") int size,
136     Model model) {
137
138     Pageable pageable = PageRequest.of(page, size);
139     Page<Product> productPage = productService.searchProducts(keyword, pageable);
140
141     model.addAttribute("products", productPage.getContent());
142     model.addAttribute("currentPage", page);
143     model.addAttribute("totalPages", productPage.getTotalPages());
144     model.addAttribute("keyword", keyword);
145
146     return "product-list";
147 }

```



Product Management System



All Categories

Exercise 6: Validation

Example:

```

<div class="form-group">
  <label for="productCode">Product Code *</label>
  <input type="text"
    id="productCode"
    th:field="*{productCode}"
    th:errorclass="error"
    required />
  <span th:if="${#fields.hasErrors('productCode')}}"
    th:errors="*{productCode}"
    class="error-message">Error</span>
</div>

```

Error display:

+ Add New Product

Product Code *

~~~~~

Product code must start with P followed by numbers

Product code must be 3-20 characters

Product Name \*

Name must be 3-100 characters

Product name is required

Price (\$) \*

Price is required

Quantity \*

Quantity is required

Category \*

Select category 

Category is required

### Description

Enter product description (optional)

 Save Product

 Cancel

## Exercise 7: Sorting

The listProducts method in ProductController was updated to accept sortBy and sortDir request parameters.

```
Sort sort = Sort.unsorted();
if (sortBy != null) {
    sort = sortDir.equals("asc")
        ? Sort.by(sortBy).ascending()
        : Sort.by(sortBy).descending();
}
```

If the client specifies a field (e.g., name, price) and direction (asc or desc), the controller passes this Sort object to the service.

## Exercise 8: Statistic Dashboard

The /dashboard endpoint in ProductController aggregates all required statistics and passes them to the Thymeleaf view:

```
@GetMapping("/dashboard")
public String showDashboard(Model model) {
    model.addAttribute("totalProducts", productService.getAllProducts().size());
    model.addAttribute("totalValue", productService.calculateTotalValue());
    model.addAttribute("averagePrice", productService.calculateAveragePrice());

    // Prepare data for pie chart
    List<String> categories = productService.getAllCategories();
    List<Long> counts = categories.stream()
        .map(productService::countByCategory)
        .toList();

    model.addAttribute("categoryLabels", categories);
    model.addAttribute("categoryData", counts);

    model.addAttribute("lowStockProducts", productService.findLowStockProducts(10));

    List<Product> recentProducts = productService.getAllProducts().stream()
        .sorted((a, b) -> b.getCreatedAt().compareTo(a.getCreatedAt()))
        .limit(5)
        .toList();
    model.addAttribute("recentProducts", recentProducts);

    return "dashboard";
}
```

- Dashboard consolidates multiple metrics in one view for fast decision-making.
- Separation of concerns is maintained: repository handles data queries, service layer manages business logic, and controller prepares the model for the view.

- Chart.js integration provides a visually intuitive representation of category distribution.

